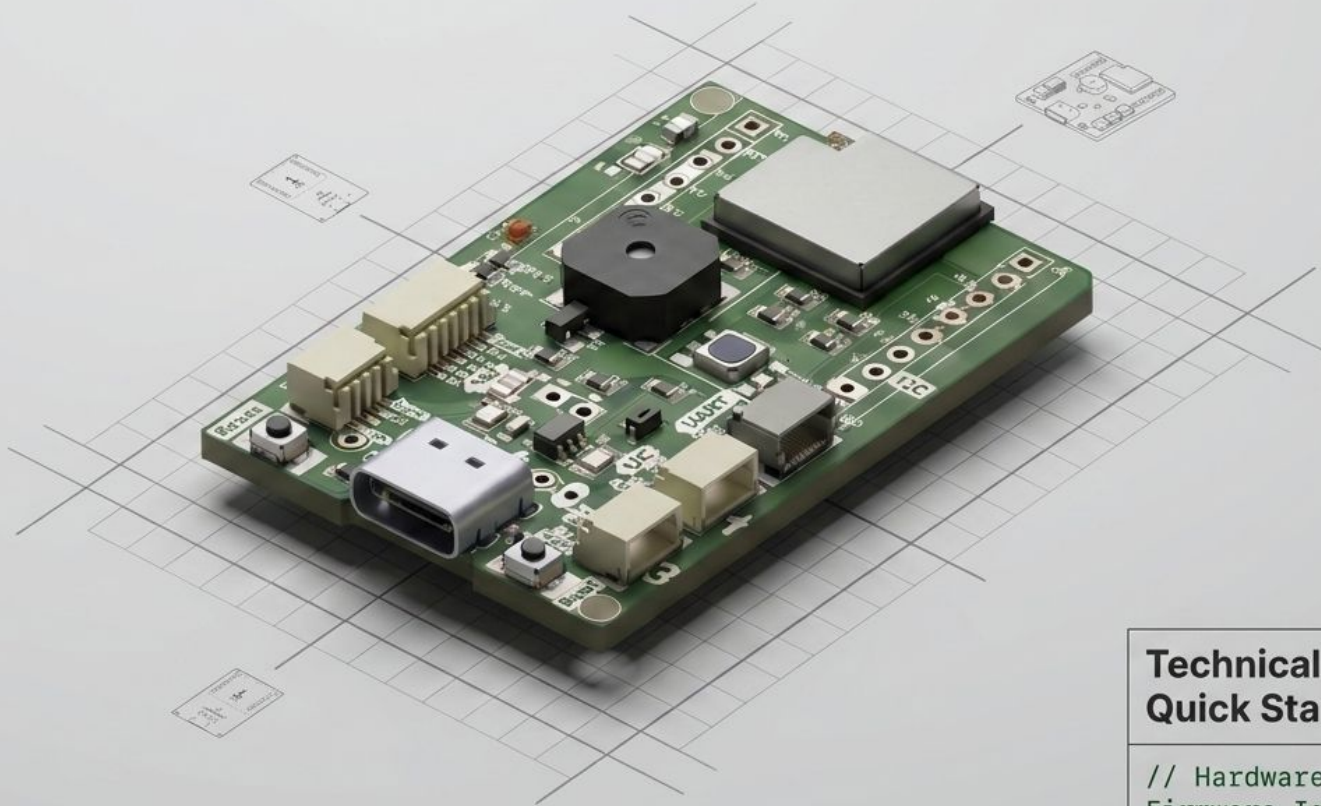


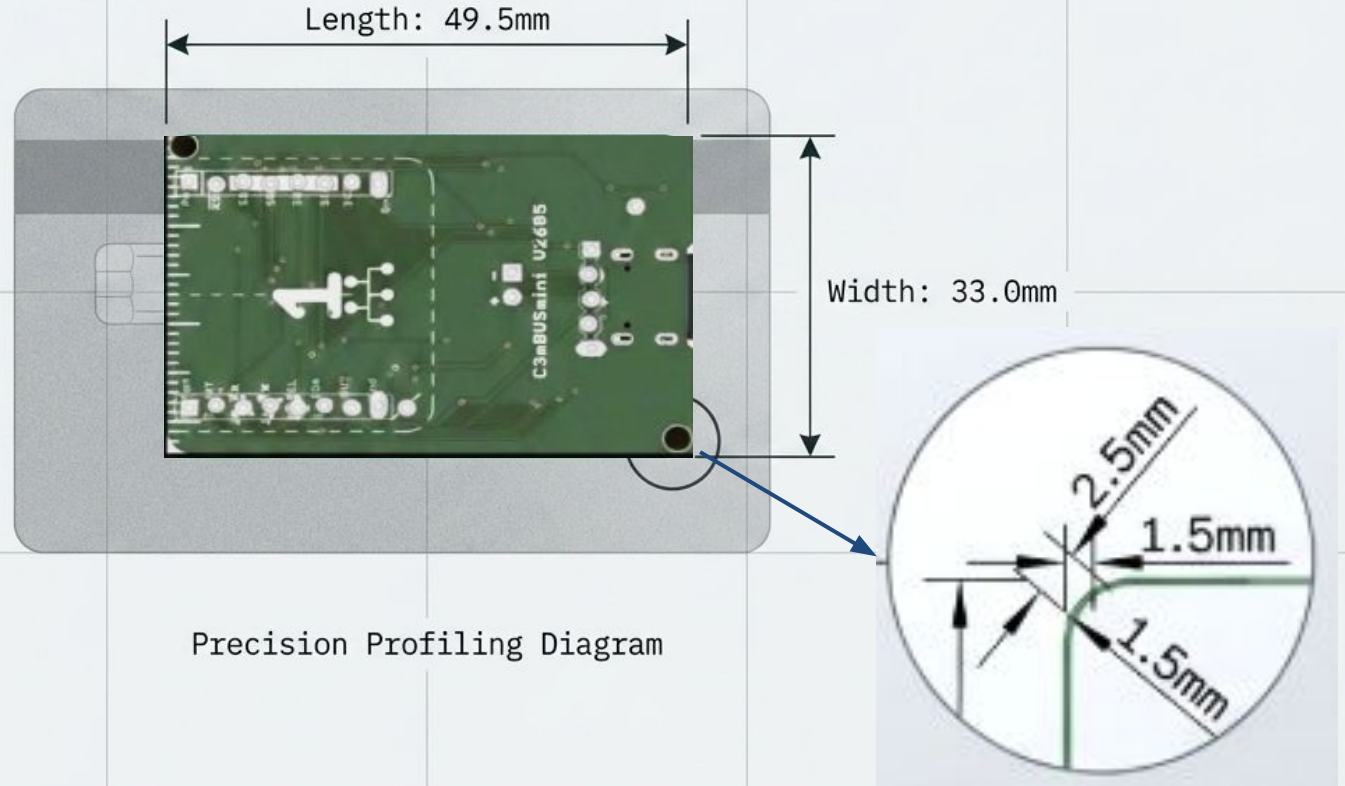
Engineering the mBusC3mini V2605



**Technical Primer &
Quick Start Guide.**

// Hardware Anatomy &
Firmware Ignition Guide

Desktop processing power scaled down to a pocket footprint



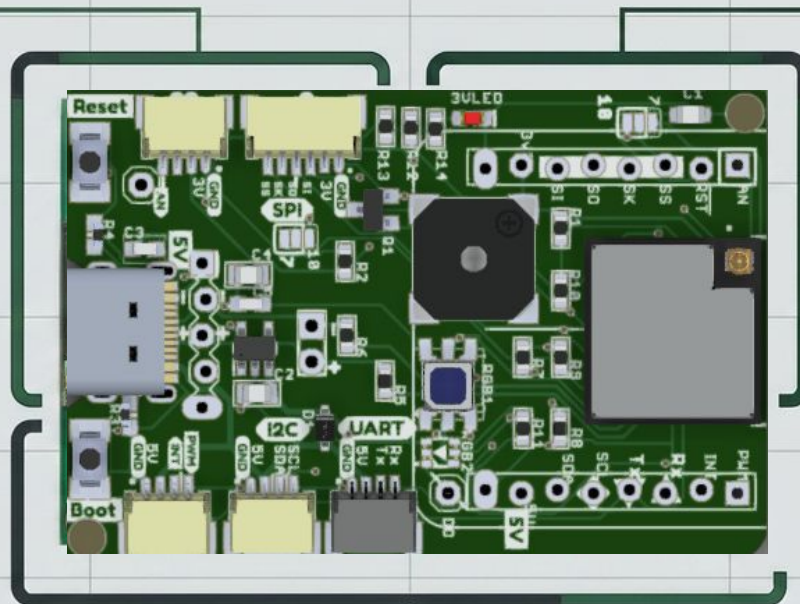
A modular architecture organized for rapid prototyping

The Voice (Feedback)

Onboard visual and auditory status indicators.

The Brain (Processing & Power)

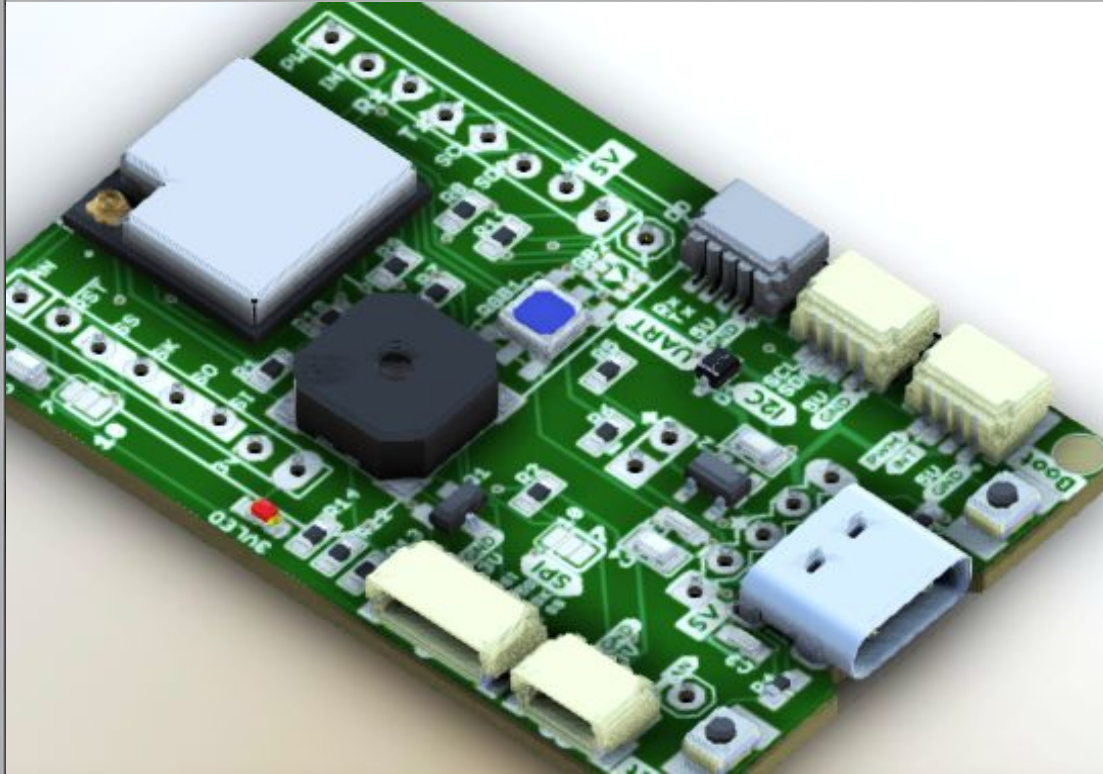
The core computing unit and voltage regulation.



The Senses (I/O & Connectivity)

Micro-connectors for peripherals and data busses.

Driven by the ESP32-C3 architecture



Compute
Engine:

ESP32-C3-mini-1U

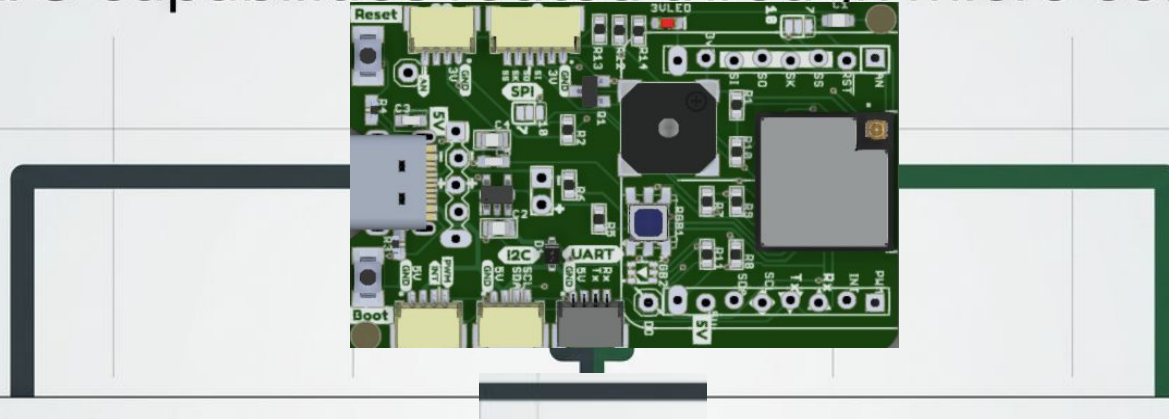
Memory &
RF:

4MB onboard Flash
memory paired
with a dedicated
u.FL antenna
socket for
reliable wireless
range.

Power
Delivery:

USB Type-C port
(Power/CDC)
routed through a
dedicated 3.3V
regulator for
stable voltage
under load.

Massive I/O capabilities routed through micro connectors



Connectivity Matrix

JST-1mm-6p connector



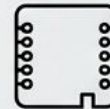
Dedicated high-speed
SPI port.

JST-1mm-4p connectors



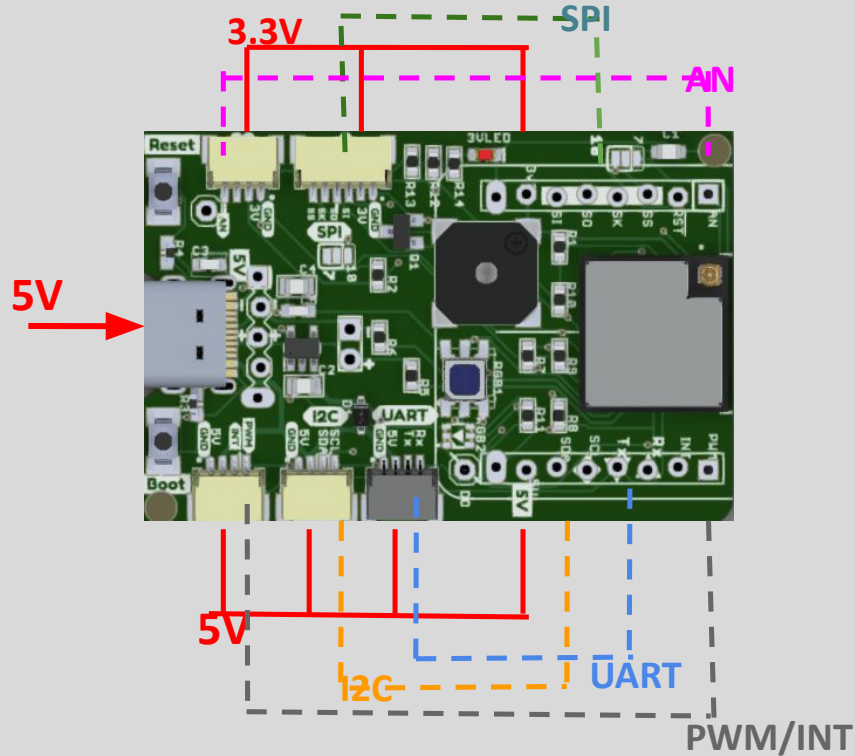
Dedicated ports for **UART**,
I2C, **PWM**, and **AN**.

Expansion

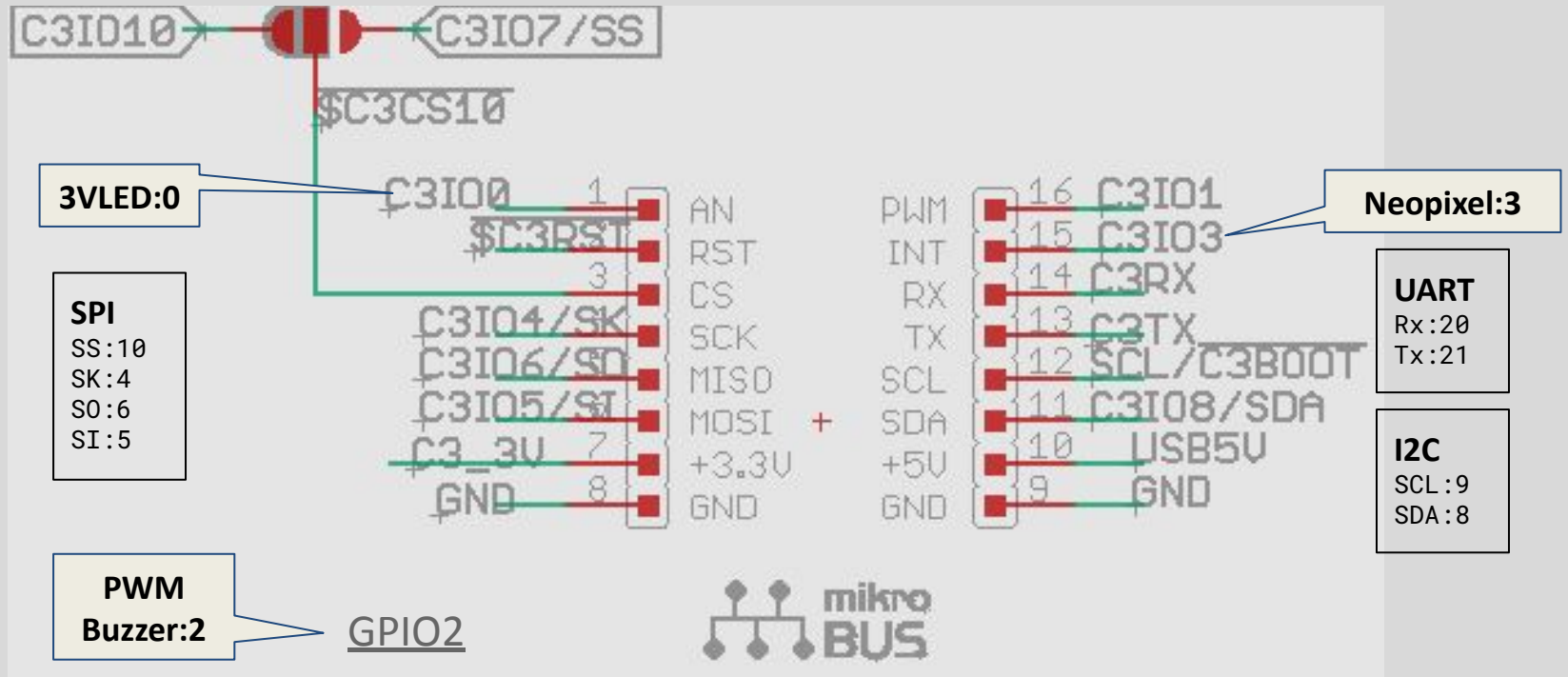


Standard **mikroBus**
connector for modular
snap-in peripheral boards.

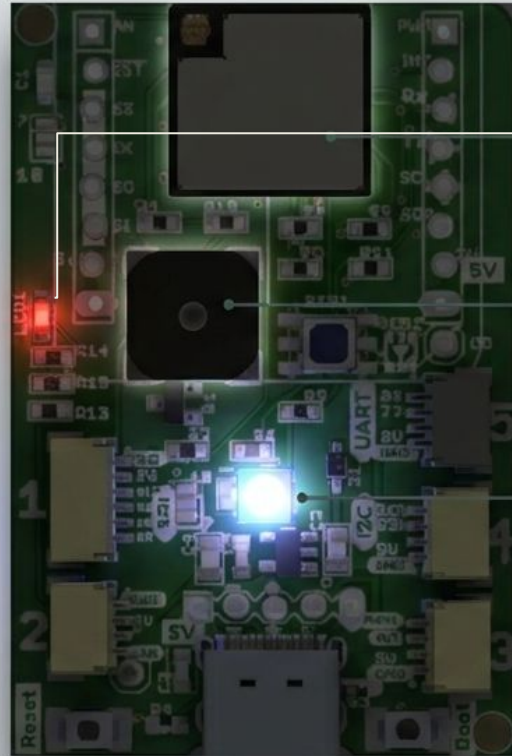
mikroBus® / JST®-SH connectors shared pins



mikroBus®/ MCU mapped pins



Immediate hardware feedback mechanisms for active debugging



Status Indicators

3VLED (AD0) for binary status.

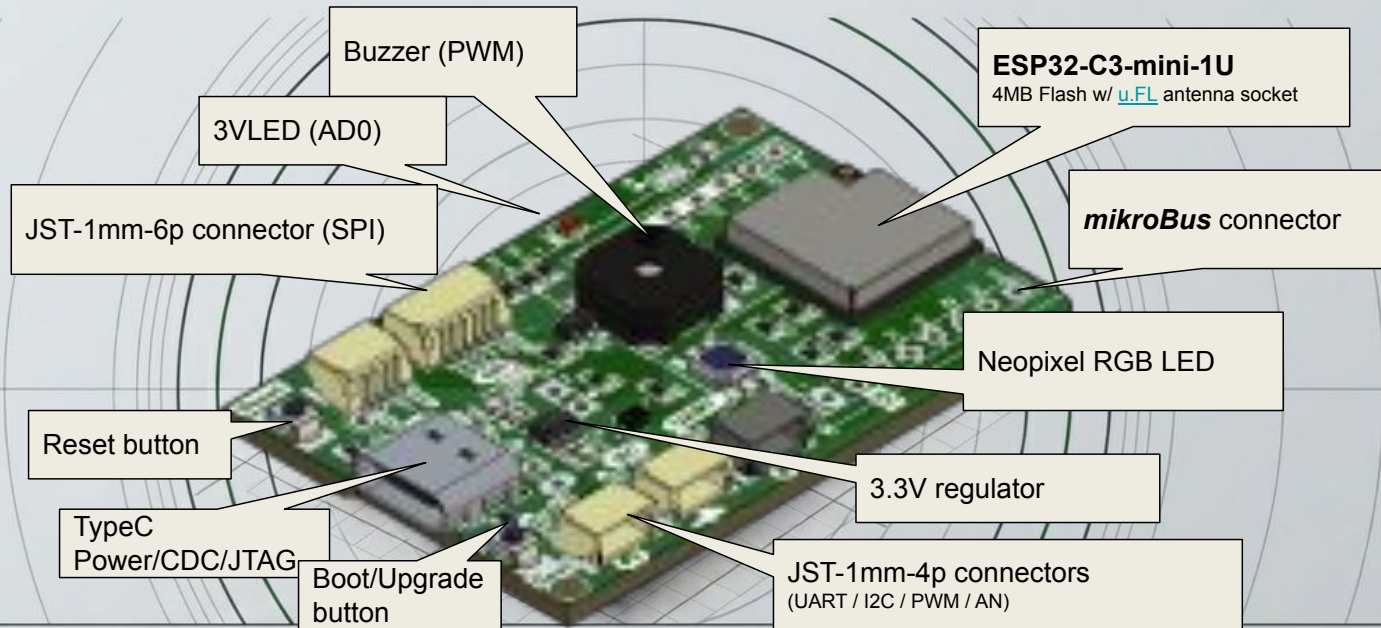
Acoustic

Onboard Buzzer (PWM) for auditory state alerts.

Programmable Feedback

Fully programmable Neopixel RGB LED for complex state visualization.

A complete prototyping ecosystem ready for deployment



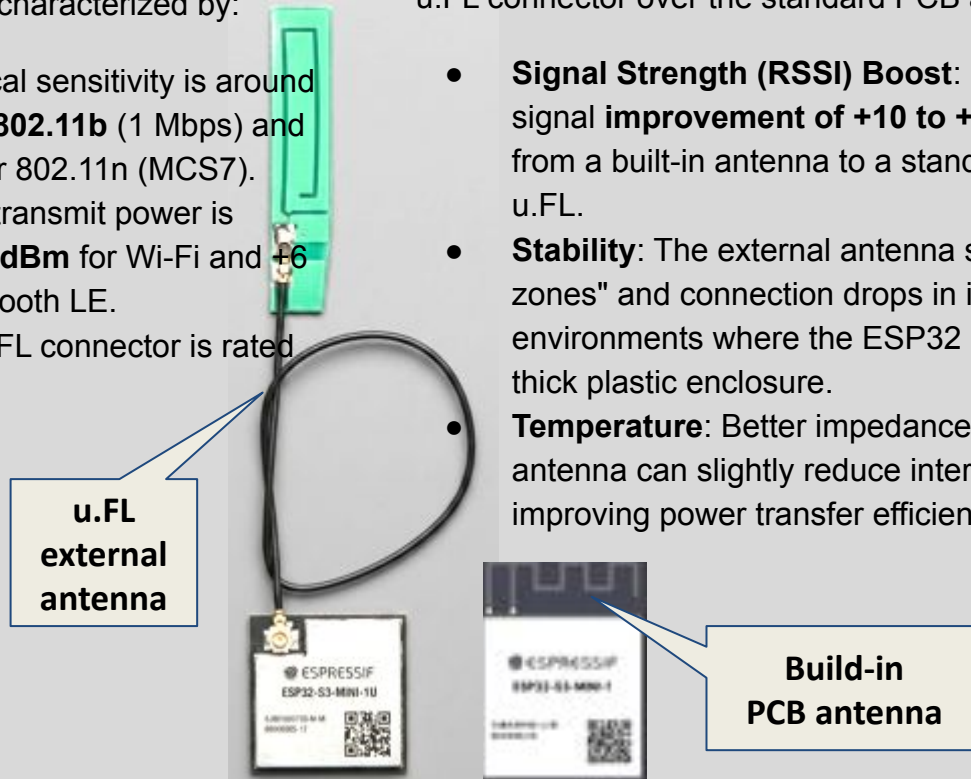
The mBusC3mini consolidates desktop-grade ESP32-C3 computing, standardized mikroBus expansion, and instant state feedback into a 49.5mm footprint. It is a complete, self-contained staging ground. The hardware is primed. It is time to ignite the software.

About the RF performance

Official RF Performance (Espressif®)

According to the official Espressif Systems Datasheet, the module's RF performance is characterized by:

- **Receiver Sensitivity:** Typical sensitivity is around **-97 dBm to -98 dBm for 802.11b** (1 Mbps) and approximately -76 dBm for 802.11n (MCS7).
- **Output Power:** Maximum transmit power is typically **+18 dBm to +20 dBm** for Wi-Fi and +6 dBm to +10 dBm for Bluetooth LE.
- **Target Impedance:** The u.FL connector is rated for **50 Ω** impedance.

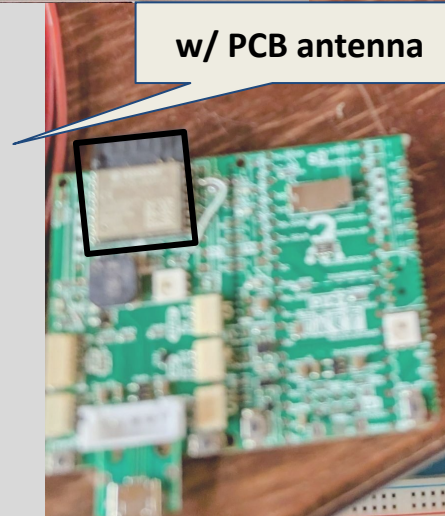
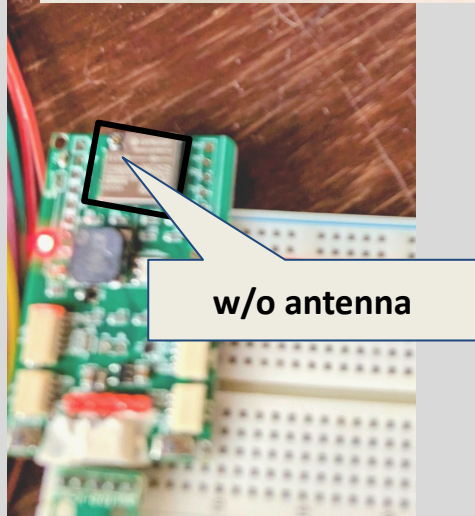
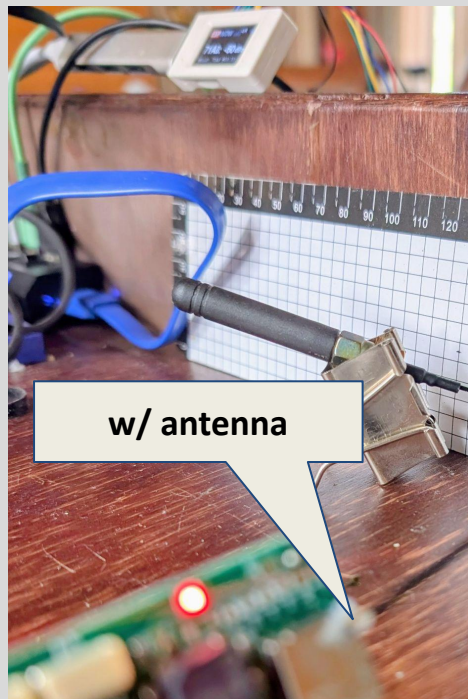


u.FL vs. PCB Antenna Comparison

Community benchmarks, highlight the following when using the u.FL connector over the standard PCB antenna:

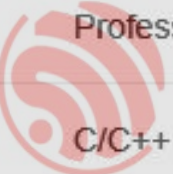
- **Signal Strength (RSSI) Boost:** Users frequently report a signal **improvement of +10 to +15 dBm** when switching from a built-in antenna to a standard dipole antenna via u.FL.
- **Stability:** The external antenna significantly reduces "dead zones" and connection drops in industrial or smart home environments where the ESP32 might be inside a metal or thick plastic enclosure.
- **Temperature:** Better impedance matching with an external antenna can slightly reduce internal chip temperature by improving power transfer efficiency.

ESPnow® Demo



Software Development Environments

The most common choices are the beginner-friendly Arduino IDE, the versatile VS Code with PlatformIO, and the official ESP-IDF.

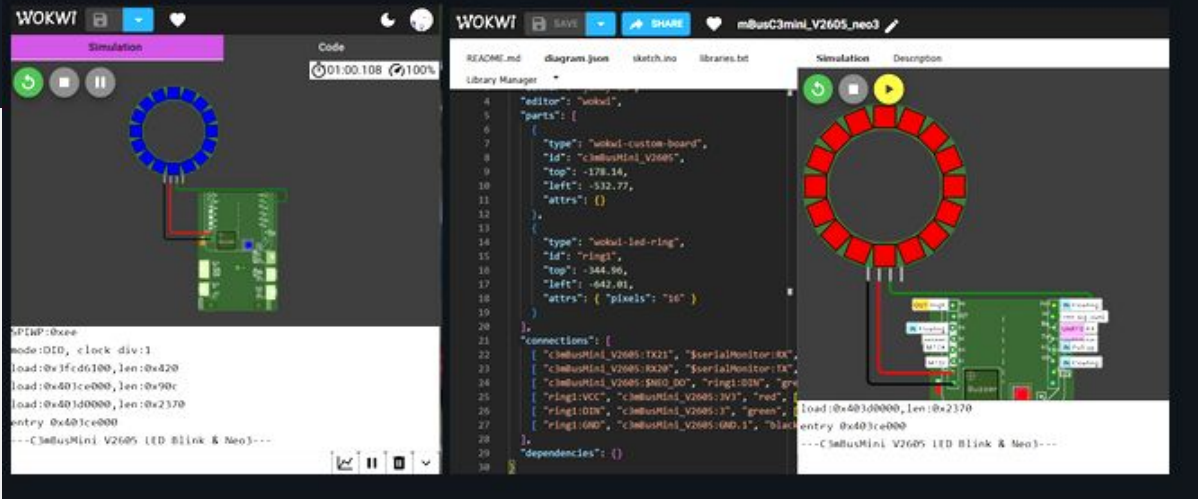
Feature 	Arduino IDE	VS Code + PlatformIO	ESP-IDF
Learning Curve	Very Low	Moderate	High
Flexibility	Low Arduino IDE	High PlatformIO	Full ESP-IDF
Project Management	Basic  C++ (Arduino)	Advanced  C/C++ 	Professional  C/C++
Primary Language			
	Ease of Use		
	Full Control		

Create and simulate Wokwi® parts in browser

Simple LED Blink

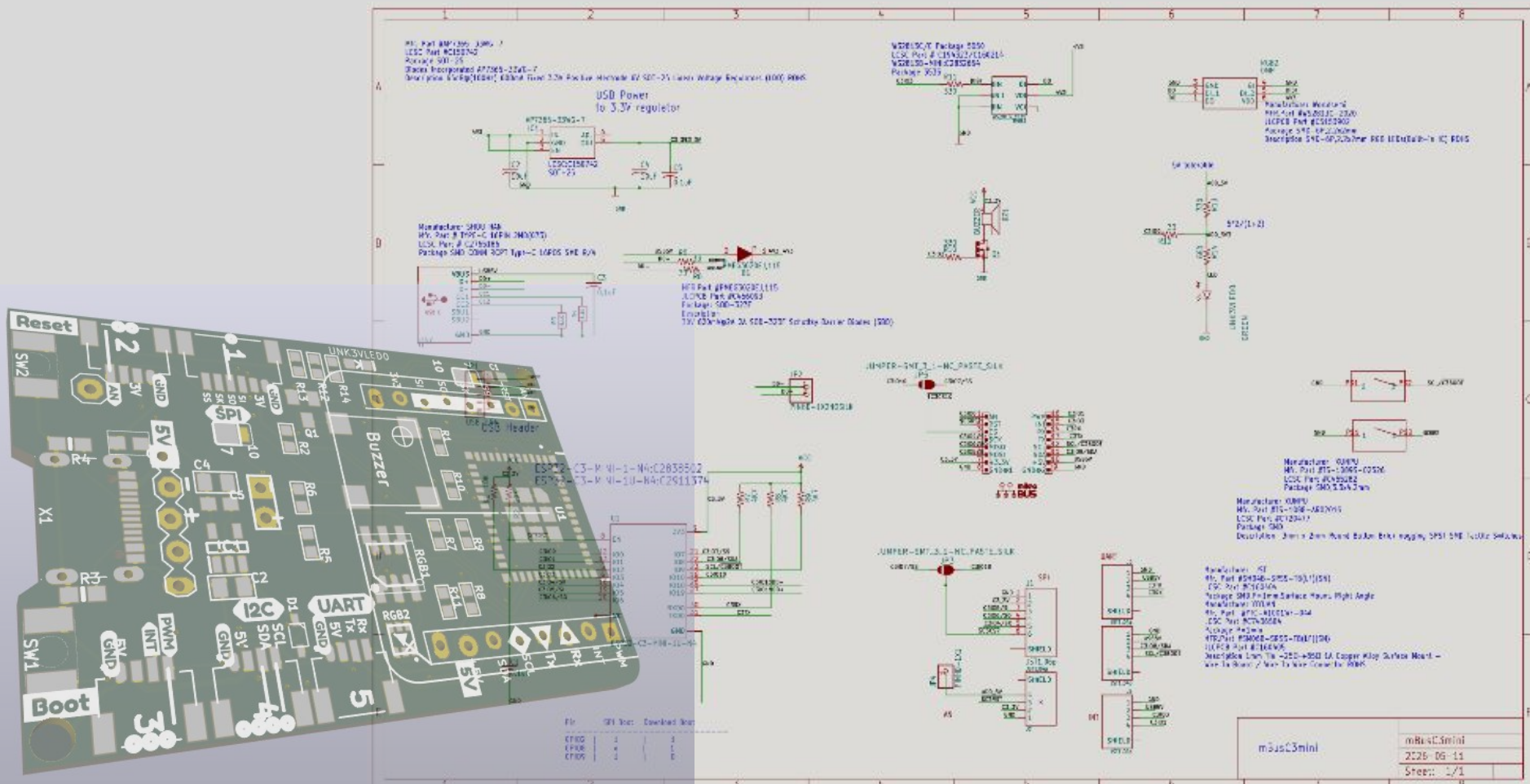


Neopixels and LED Blink



Wokwi® custom board : <https://github.com/jmysu/mBusC3mini/tree/main/WokwiSimulator>

Open source hardware at Github®



pinConfig, ESP32Info

CLI, ESPnow

```
Firmware Ver:1.0.2605
Build:09:38:36 May 11 2026
help
```

```

Commander CLI
#00 User Commands:
#
#-----Ods-----
# reset soft reset
# info show OHP info...
# gpio gpio <pin>,[0|1]; read/set gpio pin
# listpin listpin_list_mypinconfig NMS data
# blink blink <ms>, <ms ~ 30>; (<30 to turn off)
# no no <cr>,<g>,<b>; set neopixel rgb color
# neoblink neoblink <cr>; enable/disable neopixel blink
# play play [up/down/over/pack|135|53|crazy|marrio|titanic|pirate]
# beep beep <dur>,<ms>
# cq cq [string]
# listfil list files on filesystem
#-----I2C-----
# sda sda [pinSDA], set I2C SDA pin
# scl scl [pinSCL], set I2C SCL pin
# i2cscan i2cscan, scan devices on i2c bus
# i2ctest i2ctest, test devices on i2c bus
# i2c i2c <hex> <addr>, <hex> data <addr>... write device with data
# i2cr i2cr <hex> <addr>, <hex> data write device register
# i2cr i2cr <hex> <addr>, <hex> read device register
#-----ESPnow-----
# now now [i|0], enable/disable ESPnow traffic messages
# naming naming <str>, send ESPnow message
#-----
# QD>

```

```
CMD>now 1
ESPnow traffic:1
CMD>
PlainText sent (17)----->>>
MsgPack sent (26)>>>>>>>>>
83 A2 69 64 C0 15 1C A4 74 65 6D 70 CA 42 3E C0
C0 A7 63 6F 75 6E 74 65 72 01
```

```
Received: (Non-MsgPack) <<<------(25)
HELLO! 0x3DC4F4 @00:00:53
```

48|45|4C|4C|4F|21|20|30| 78|33|44|43|34|46|34|20|
40|30|30|3A|30|30|3A|35| 33|

RSSI: -26 dBm (Excellent)
From: 48:27:e2:3d:c4:f4
Broadcast

```
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
Received: (MsgPack data) <<<<<<<<<(26)
ID16: 0xC4F4
Chip Temp: 38.6°C
Counter: 5
```

83	A2	69	64	CD	C4	F4	A4	74	65	6D	70	CA	42	1A	66
66	A7	63	6F	75	6E	74	65	72	05						

RSSI: -26 dBm (Excellent)
From: 48:27:e2:3d:c4:f4
Broadcast

[illegible]

Ready to use codes: pinConifg, CLI, ESPnow, I2C scanner, PWM buzzer, Neopixel LED, TinyBasic+...

<https://github.com/jmysu/mBusC3mini/tree/main>

Easy firmware upgrade

From unboxing to running code in three linear stages



Stage 1: Initiate hardware flash mode

User Action



Action: (Optional) Hold the B00T key, then press the RESET key.

System State

```
ESP-ROM:esp32c3-20200918 Build:Sep 18 2020
rst:0x1 (POWERON),boot:0x5 (DOWNLOAD(USB/UART0/1))
waiting for download_
```

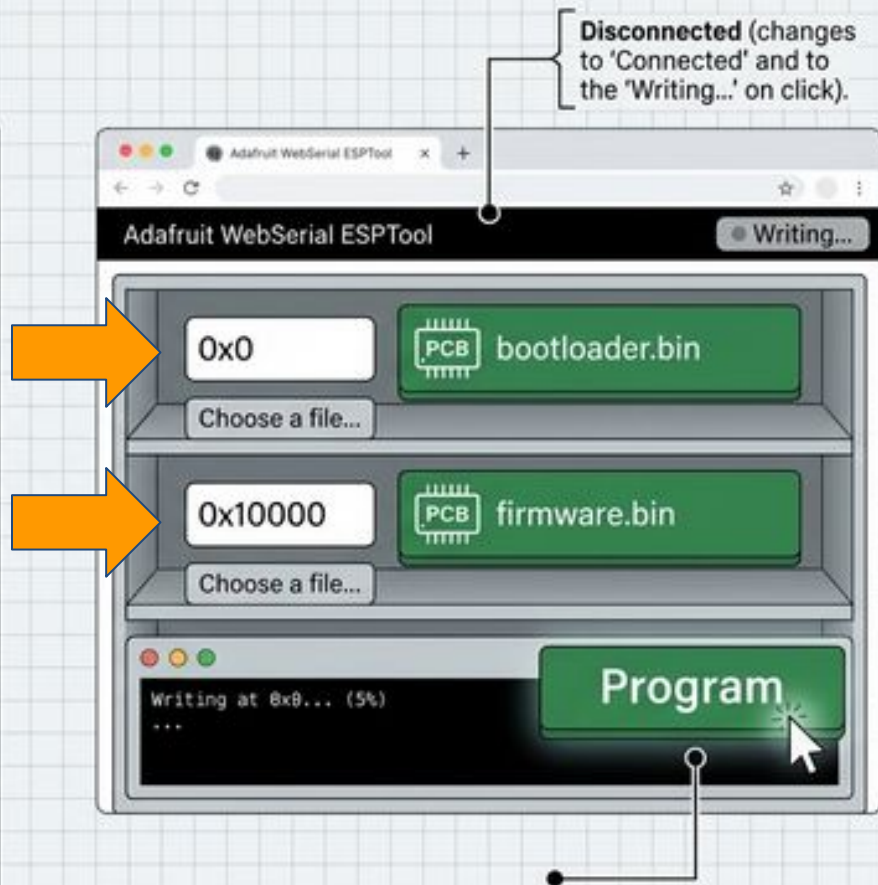
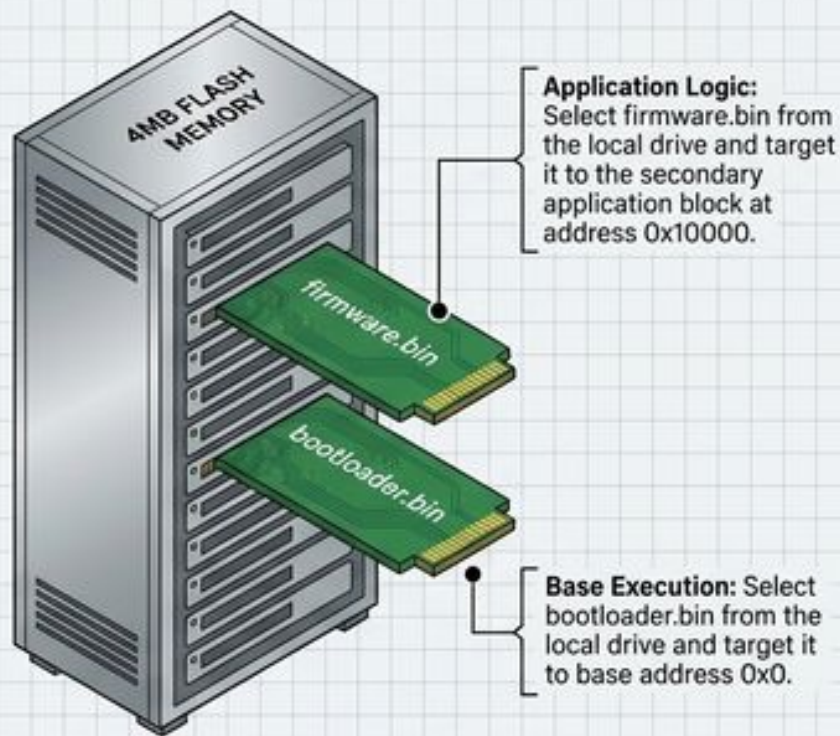
State: The ESP board halts the current execution loop and enters the bootloader flash state, waiting for serial instructions.

Stage 2: Establish the browser bridge via WebSerial



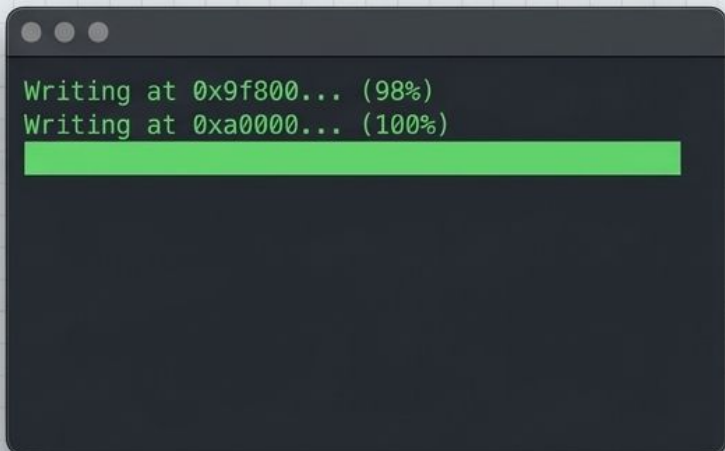
Environment	Target Tool	Execution
Launch ESPTool directly from a Chrome or Edge browser.	Navigate to the Adafruit WebSerial ESPTool.	Select the matching USB/ USB/Serial debug port and hit 'Connect' to link the browser directly to the ESP board.

Stage 3: Map the binaries directly to flash memory



Execute the write protocol and ignite the board

Digital State

A terminal window with a dark background and three window control buttons in the top-left corner. It displays two lines of green text: "Writing at 0x9f800... (98%)" and "Writing at 0xa0000... (100%)". Below the text is a solid green horizontal bar representing a progress indicator.

```
Writing at 0x9f800... (98%)  
Writing at 0xa0000... (100%)
```

Flash: Press PROGRAM to begin the upgrade process. The terminal will output precise writing addresses and percentage completions.

Physical Action



Ignition: Once the update concludes, press the physical RESET button one final time to restart the board and execute the newly deployed firmware.